PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation

PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation
- 作者前言
- Abstract（摘要）
- 1. Introduction（引言）
- 2. Related Work（相关工作）
- 3. Problem Statement（问题陈述）
- 4. Deep Learning on Point Sets（针对点集的深度学习）
  - 4.1 Properties of Point Sets in $\mathbb{R}^n$
  - 4.2 PointNet Architecture（PointNet架构）
    - 适用于无序输入的对称函数
    - 局部和全局信息的聚合
    - Joint Alignment Network
    - 联合对齐网络

作者前言

近十年来，深度学习技术在解决各领域问题方面展现出了强大的能力，包括三维数据处理领域。由于二维图像数据在深度信息上的丢失，很多对精度要求高的任务必须使用三维数据，如三维重建、自动驾驶，以及其延伸出的点云对齐、三维实体识别、三维语义分割等，这就为深度学习的研究者们提供了很大的研究空间。本文提出了一种具有通用性的模型结构PointNet，它对点云格式的数据进行处理，能被复用与分类、分割等多个领域。由于点云数据在结构上的特殊性，传统的深度学习网络无法直接应用，因此作者对模型进行了重新设计，理论上证明了PointNet的泛化能力，并通过可视化解释了它的工作机制。该模型在多个3D任务上达到了与或超过现有方法的性能，展示了其在实际应用中的潜力。

本文假设读者对点云和其他常见三维表示的原理、特点、操作有基本的认识，对此不了解的读者可以自行查找相关介绍文章。下面我们就按原文进行逐段的翻译和详解。

Abstract（摘要）

> Point cloud is an important type of geometric data structure. Due to its irregular format, most researchers transform such data to regular 3D voxel grids or collections of images. This, however, renders data unnecessarily voluminous and causes issues. In this paper, we design a novel type of neural network that directly consumes point clouds, which well respects the permutation invariance of points in the input. Our network, named PointNet, provides a unified architecture for applications ranging from object classification, part segmentation, to scene semantic parsing. Though simple, PointNet is highly efficient and effective. Empirically, it shows strong performance on par or even better than state-of-the-art. Theoretically, we provide analysis towards understanding what the network has learned and why the network is robust with respect to input perturbation and corruption.

翻译：① 点云是一种重要的几何数据结构。② 由于其不规则格式，大多数研究者将这类数据转换为规则的3D体素网格或图像集合。然而，这样的转换往往导致数据冗余，造成了一些问题。③ 在本文中，我们设计了一种新的神经网络，能够直接处理点云，同时很好地尊重输入点的置换不变性。④ 我们的网络名为PointNet，提供了一个统一的架构，适用于从物体分类、部件分割到场景语义解析等多个应用。⑤ 尽管结构简单，PointNet却非常高效且有效。实证结果表明，其性能与最先进的技术相当，甚至更优。从理论上，我们提供了对网络学习内容的分析，解释了该网络为何能对输入扰动和损坏保持鲁棒性。

解释：

- ①句提到作者首先提到点云是一种“**不规则**”的数据格式，那么，何为“不规则”，“规则”又是什么呢？事实上，这里的规则可以理解为**有固定的网格**。例如体素格式，它有固定的分辨率、固定的网格大小，指定起点坐标后就可以在空间中指定出一个个排列规则的小方格，就像图像的像素一样。这种规则排列又带来另一个性质，即**有序**。显然如果我们使用索引`(i, j, k)`分别访问两个体素网格，那么索引出来的体素一定是相对应的。而点云数据结构没有固定的网格，它可以占用空间中的任意一个连续位置，它也没有顺序，两个点云之间的各个点也没有明确的对应关系，这就是所谓的**不规则**。如果要用更精确的几个词来描述这个性质的话，可以概括为**连续** (continuous)、**稀疏** (sparse)、**无序** (unordered)。
- ②句提到，以往的工作往往会实现将不规则的点云格式转为体素网格格式，甚至采样为图片进行处理，以适应需要规则输入的神经网络。但这就导致了数据的冗余。可以设想一下，点云作为一种稀疏的数据结构仅需要保存物体部分的数据即可，空白部分是不占用空间的，但体素网格却要保存所有位置的信息。再考虑到N^3的增长速率，一个$128 \times 128 \times 128$的网格，就需要差不多200万个体素。一个直观的例子是，笔者的一个项目中，点云表示平均3~5MB，而网格则需要500MB，这对于内存、显存、IO来说都形成了巨大的压力。可见，这类转换确实会带来巨大的问题。
- ③句提到，该网络尊重了输入点的**置换不变性**（permutation invariance）。**不变性**（Invariance）是指某个系统、函数或模型在输入发生某种特定变化（例如平移、旋转、缩放或排列）时，其输出保持不变的性质。相信读者如果学习过计算机视觉相关的课程应该不难理解，一个简单的例子是，好的图像识别算法应该具有**平移不变性**（translation invariance），因为物体的类别显然不取决于它出现在图像的什么位置。置换不变性指输出和输入的顺序无关，例如对于函数$f(x)$，如果输入$x=(a,b,c)$，a,b,c的位置可任意互换，输出都是一致的，则$f(x)$具有置换不变性。
- ④句提到，PointNet适用于多种下游任务。这说明PointNet架构不仅仅关注点云某方面的特征，而是能够较为全面地提取包括局部结构、全局结构、语义等各个层面的特征。假设提取出来的特征可以用一个特征向量表示的话，那么我们仅需要训练一个新的输出层就可以实现不同的功能。

---

1. Introduction（引言）

> In this paper, we explore deep learning architectures capable of reasoning about 3D geometric data such as point clouds or meshes. Typical convolutional architectures require highly regular input data formats, like those of image grids or 3D voxels, in order to perform weight sharing and other kernel optimizations. Since point clouds or meshes are not in a regular format, most researchers typically transform such data to regular 3D voxel grids or collections of images (e.g., views) before feeding them to a deep net architecture. This data representation transformation, however, renders the resulting data unnecessarily voluminous—while also introducing quantization artifacts that can obscure natural invariances of the data.

翻译：在本文中，我们探讨了能够推理3D几何数据（如点云或网格）的深度学习架构。① 典型的卷积架构需要高度规则的输入数据格式，例如图像网格或3D体素，以便进行权重共享和其他核优化。② 然而，由于点云或网格并不是规则格式，大多数研究者通常会在将其输入深度网络架构之前，将这些数据转换为规则的3D体素网格或图像集合（例如视图）。③ 然而，这种数据表示转换使得结果数据不必要地冗长，同时还引入了量化伪

影，这可能会掩盖数据的自然不变性。

解释：

- ①句已经进行了部分解释，这里再说明下为什么高度规则的输入数据格式才能实现权重共享和其他核优化。**权重共享**（weight sharing）是指在深度学习中，卷积核的参数（权重）在输入的不同位置重复使用，从而减少参数数量。只有规则的输入（比如图片像素网格或3D体素）才能实现权重共享，因为它们的结构在不同的位置是一致的，是不变的，卷积核在任意位置都能匹配到相同大小、相同排列的局部区域，因此可以使用一个固定的规则对不同位置的信息进行处理。而点云的排列是任意的，不存在一个能同时处理任意位置的通用方法，也就无法实现权重共享。
- ②句在摘要部分已经进行了说明。这里解释一下③句中的**量化伪影**（quantization artifacts）。举一个通俗的例子，假设有一个点云$(1.4, 1.6, 2.4)$，那么如果把它规范化到一个网格中，比如中心坐标为$(1, 2, 2)$的网格中，这就使原来的三维结构产生了偏差。在分辨率不足的情况下，可能多个点云合并到同一个网格内，造成细节丢失和模糊；可能一条线上连续的点云被划分到不同网格，形成锯齿状结构。

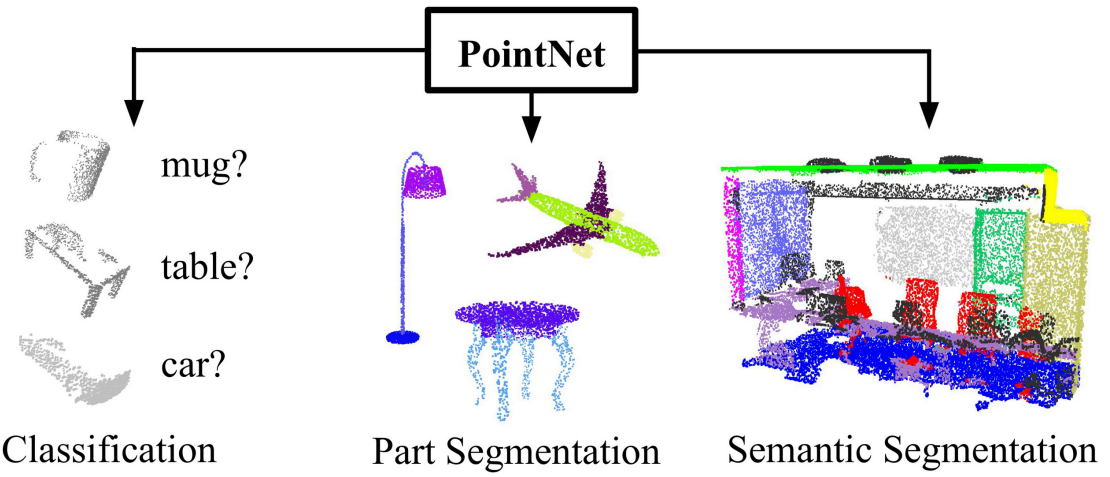


Figure 1. Applications of PointNet. We propose a novel deep net architecture that consumes raw point cloud (set of points) without voxelization or rendering. It is a unified architecture that learns both global and local point features, providing a simple, efficient and effective approach for a number of 3D recognition tasks.
图1. PointNet的应用。我们提出了一种新颖的深度网络架构，它能够直接处理原始点云（点集），无需体素化或渲染。 这是一种统一的架构，能够学习全局和局部点特征，为多种3D识别任务提供了一种简单、高效且有效的方法。

> For this reason, we focus on a different input representation for 3D geometry using simply point clouds —and name our resulting deep nets PointNets. Point clouds are simple and unified structures that avoid the combinatorial irregularities and complexities of meshes, and thus are easier to learn from. The PointNet, however, still has to respect the fact that a point cloud is just a set of points and therefore invariant to permutations of its members, necessitating certain symmetrizations in the net computation. Further invariances to rigid motions also need to be considered.

翻译：因此，我们专注于使用点云作为3D几何的不同输入表示，并将我们所得到的深度网络称为PointNets。点云是简单且统一的结构，避免了网格的组合不规则性和复杂性，因此更容易进行学习。然而，PointNet仍然必须考虑到点云仅仅是一组点，因此对其成员的排列是不变的，这就需要在网络计算中进行某些对称化。此外，还需要考虑对刚体运动的不变性。

解释：

- 这段话主要是说明了PointNet的不变性，它对于刚体运动（平移、旋转）是鲁棒的，点云的位置和姿态不会影响网络的输出。同时PointNet还具有置换不变性，点云的顺序也不会影响到输出。PointNet对每个点进行独立的特征提取，然后使用对顺序不敏感的方式对这些独立特征进行聚合形成全局特征，从而保障输出和输入点顺序无关。这部分原理后面再做介绍。

> Our PointNet is a unified architecture that directly takes point clouds as input and outputs either class labels for the entire input or per point segment/part labels for each point of the input. The basic architecture of our network is surprisingly simple, as in the initial stages, each point is processed identically and independently. In the basic setting, each point is represented by just its three coordinates (x, y, z). Additional dimensions may be added by computing normals and other local or global features.

翻译：我们的PointNet是一个统一的架构，能够直接将点云作为输入，并输出整个输入的类别标签或每个点的分段/部分标签。我们的网络基本架构出奇简单，因为在初始阶段，每个点都是独立且相同地处理。在基本设置中，每个点仅由其三个坐标(x, y, z)表示。通过计算法线和其他局部或全局特征，可以添加额外的维度。

解释：

- 这一段介绍了PointNet的输入输出。输入就是一个点云，每个点由三个坐标分量描述，数据的维度为`(N, 3)`。在Open3D库中，对一个点云对象获取其`points`属性，打印`pcd.points.shape`，输出正是`(N, 3)`，其中N是点的数量。
- 输出则根据任务的不同有不同的形态。总的来说，在全局特征聚合后，网络会输出一个全局特征向量，这一步过后网络会开始分化。对于分类任务，这个向量会通过一系列全连接层直接转化为对类别的预测。对于分割任务，则会将每个点的局部特征和全局特征向量分别拼接，形成新的更长的向量，然后这些向量会分别通过一个多层感知机转化为各个点的类别得分。关于输出的具体维度和使用方法，我们后面再做介绍。

> Key to our approach is the use of a single symmetric function, max pooling. Effectively, the network learns a set of optimization functions/criteria that select interesting or informative points of the point cloud and encode the reason for their selection. The final fully connected layers of the network aggregate these learned optimal values into the global descriptor for the entire shape as mentioned above (shape classification) or are used to predict per point labels (shape segmentation).

翻译：① 我们方法的关键是使用一个单一的对称函数——最大池化。② 实际上，网络学习了一组优化函数/标准，用于选择点云中有趣或信息丰富的点，并编码其选择的原因。③ 网络的最终全连接层将这些学习到的最优值汇总为整个形状的全局描述符，如前所述（形状分类），或者用于预测每个点的标签（形状分割）。

解释：

- 前面我们提到PointNet使用了对顺序不敏感的函数对特征进行聚合，①句则介绍了这个函数的名称——**最大池化**（Max Pooling）。在PointNet中，该函数对每个特征维度求最大值，找出在这个特征下特征最显著的点，并使用这个点在该维度上的值作为全局特征在该维度上的值。
    - 举例说明：如果点p_1的特征向量为$[2, 3, 1]^T$，点p_2的特征向量为$[1, 4, 2]^T$，那么经过最大池化后生成的全局特征向量就是$[2, 4, 2]^T$。

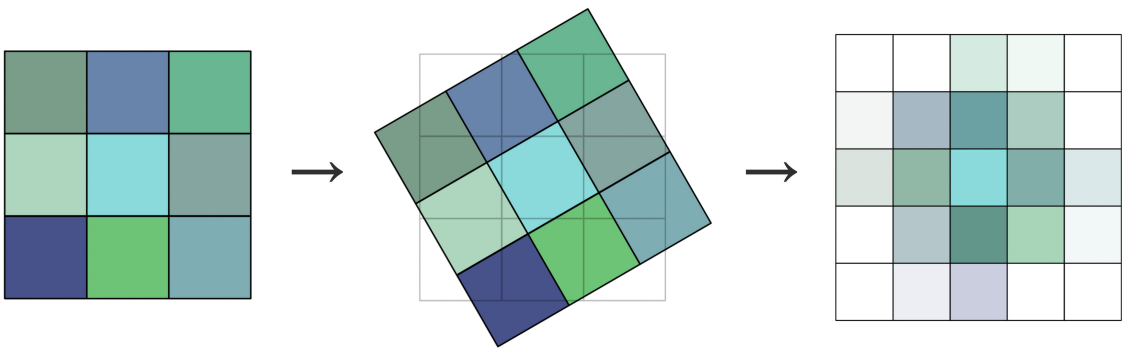
- “使用每个特征维度中最显著的点的值来描述整体特征”是一种通用且有效的思路。其核心思想是：通过最大池化操作，保留每个特征维度中最重要或最突出的信息（即最大值），这些值往往来自关键点，能够反映点云的核心几何结构和语义特征。相比于平均池化，最大池化更关注显著特征，不会被不相关的点（如噪声点或背景点）稀释掉。
- ②句提到“选择点云中有趣或信息丰富的点”，指的就是通过最大池化找到每个特征维度下该特征最显著的点，并获取其特征值。而“网络学习的一组优化函数/标准”指的是局部特征提取的过程，它对单个点的各个特征显著程度进行打分，让最大池化能够判断哪些点的特征最有价值。这个打分的操作就是原文所说的“编码其选择的原因”。
- ③句在上一部分已经提过，指的是特征向量转换到输出结果的过程。

> Our input format is easy to apply rigid or affine transformations to, as each point transforms independently. Thus, we can add a data-dependent spatial transformer network that attempts to canonicalize the data before the PointNet processes them, so as to further improve the results.

翻译：① 我们的输入格式便于对每个点独立应用刚性或仿射变换。② 因此，我们可以添加一个数据依赖的空间变换网络，在PointNet处理数据之前尝试对数据进行规范化，以进一步提高结果。

解释：

- ①句说明了使用点云的一个优势。由于点云的各个点是独立、无序的，因此只需要对每个点进行相应的变换，就可以使整体进行变换，非常简便。
- 这里读者可能有一个疑问，对每个单元进行相同的变换，不就能实现整体的变换了吗？为什么只说点云能够对每个点独立应用变换呢？事实上，基于规则采样的数据格式，如2D图像、体素网格的确都是无法做到这一点的。我们可以用2D图像举例：



  可以看到，原来的2D图像在每个单元（即像素）进行单独变换后，新的单元不再适配原有的网格了。因此，我们就必须通过线性插值等手段（上图使用双线性插值）重新确定每个网格的值，结果如上图中最右边的图像。类似地，体素网格在变换后也需要重新进行规则化，实现较为复杂。相比之下，点云在每个点单独变换后不需要任何后处理，因而最为简便，而且也不会产生任何信息丢失和偏差。
- ②句提到了一个新的**网络空间变换网络T-Net**（Spatial Transformer Network），它可以看做是PointNet的外挂组件，分别在输入点云前、提取局部特征后计算一个变换矩阵，将它们对齐到一个标准化的几何空间。这部分详细内容见后文。原文中“**数据依赖**”是指变换是根据输入数据内容动态学习得到的，而不是一个固定的或预设的变换矩阵。具体来说，T-Net 会根据输入点云或点的特征，利用神经网络自动计算出一个最优的变换矩阵。

> We provide both a theoretical analysis and an experimental evaluation of our approach. We show that our network can approximate any set function that is continuous. More interestingly, it turns out that our network learns to summarize an input point cloud by a sparse set of key points, which roughly corresponds to the skeleton of objects according to visualization. The theoretical analysis provides an understanding of why our PointNet is highly robust to small perturbations of input points as well as to corruption through point insertion (outliers) or deletion (missing data).

翻译：我们提供了对我们方法的理论分析和实验评估。① 我们展示了我们的网络能够逼近任何连续的集合函数。② 更有趣的是，结果表明，网络通过一组稀疏的关键点来总结输入的点云，这些关键点大致对应于物体的骨架，从可视化的角度来看。理论分析帮助我们理解了为什么PointNet对输入点的小扰动以及通过点插入（离群点）或删除（缺失数据）造成的损坏具有很高的鲁棒性。

解释：

- ①句如果翻译为数学语言，可以是：假设一个点集表示为 **$ S = \{ x_1, x_2, \dots, x_n \} $**，其中 $x_i \in \mathbb{R}^d$ 是一个向量，可以是点的坐标，也可以是特征向量或任意向量。点集函数 $f(S)$ 是一个连续的函数，定义在点集 S 上。PointNet 的目标是学习一个函数 $g(S;\theta)$，使得对于任意的连续点集函数 $f(S)$，存在模型参数 θ，使得$g(S;\theta)$能以任意精度近似 $f(S)$。相关内容读者可以查询**神经网络的“万能逼近定理”**。作者在后文给出了论证过程。
- ②句在前文已经进行了解释。PointNet通过选择一组稀疏的关键点对整体特征进行表示，避免了不显著点、无关点的影响，具有很强的鲁棒性。作者通过理论推导、实验、可视化等手段进行了说明。

> On a number of benchmark datasets ranging from shape classification, part segmentation to scene segmentation, we experimentally compare our PointNet with state-of-the-art approaches based upon multi-view and volumetric representations. Under a unified architecture, not only is our PointNet much faster in speed, but it also exhibits strong performance on par or even better than state of the art.

翻译：在多个基准数据集上，包括形状分类、部件分割和场景分割，我们对我们的PointNet与基于多视图和体素表示的最先进方法进行了实验比较。在统一架构下，我们的PointNet不仅速度更快，而且在性能上与最先进的方法相当，甚至更优。

> The key contributions of our work are as follows:
>
> - We design a novel deep net architecture suitable for consuming unordered point sets in 3D;
> - We show how such a net can be trained to perform 3D shape classification, shape part segmentation, and scene semantic parsing tasks;
> - We provide thorough empirical and theoretical analysis on the stability and efficiency of our method;
> - We illustrate the 3D features computed by the selected neurons in the net and develop intuitive explanations for its performance.
>
> The problem of processing unordered sets by neural nets is a very general and fundamental problem – we expect that our ideas can be transferred to other domains as well.

翻译：

我们工作的关键贡献如下：

- 我们设计了一种新颖的深度网络架构，适用于处理3D中的无序点集；

- 我们展示了如何训练这样的网络以执行3D形状分类、形状部件分割和场景语义解析任务；
- 我们对方法的稳定性和效率进行了深入的实证和理论分析；
- 我们展示了网络中选定神经元计算的3D特征，并为其性能发展出直观的解释。

处理无序集合的神经网络问题是一个非常普遍和基础的问题——我们期望我们的想法能够转移到其他领域。

解释：

- 可以看到，这篇论文不仅十分具有开创新，涉及的内容也是非常全面的。它不仅提出了一种新的点云处理结构，而且论证了它能够泛化到所有无序集合问题中。这篇论文也对PointNet的原理给出了分析，为我们理解这篇论文提供了很大的便利。

---

2. Related Work（相关工作）

> **Point Cloud Features**: Most existing features for point clouds are handcrafted towards specific tasks. Point features often encode certain statistical properties of points and are designed to be invariant to certain transformations, which are typically classified as intrinsic [2, 24, 3] or extrinsic [20, 19, 14, 10, 5]. They can also be categorized as local features and global features. For a specific task, it is not trivial to find the optimal feature combination.

翻译：

点云特征：① 大多数现有的点云特征是人工设计的，针对特定任务进行优化。点云特征通常编码了点的某些统计属性，并且设计时考虑了对某些变换的不变性，②这些特征通常被分为内在特征（intrinsic）[2, 24, 3]和外在特征（extrinsic）[20, 19, 14, 10, 5]。③ 它们还可以分为局部特征（Local features）和全局特征（Global features）。对于一个特定任务，找到最佳的特征组合并非易事。

解释：

- ①句说明了传统方法的两个局限，一是人工设计，而是专用性。与之对比，PointNet通过神经网络进行自动特征提取，且可以适用于多种下游任务。这一句还说明了特征选择的一些原则，比如对不变性的满足。
- **内在特征** vs **外在特征**：内部特征与物体自身的几何属性相关，是物体的内部属性，如形状、曲率等；外部特征和坐标系相关，例如物体的位置、姿态等，显然这些都是相对于一个外部的参考系而言的。
- **局部特征** vs **全局特征**：局部特征主要是针对单个点及其领域的，比如法向量、局部形状等，更加注重局部的细节；全局特征关注点云整体的一些特性，例如几何中心、形状、大小等。这里需要注意，点云有一个特性，就是全局特征几乎相同的两个点云，局部特征不一定相同（也和局部特征提取的算法是否鲁棒有关），例如，一个相同的正方体，微观上可以使用完全不同的点云分布进行描述。

> **Deep Learning on 3D Data**： 3D data has multiple popular representations, leading to various approaches for learning.
>
> - **Volumetric CNNs**: [28, 17, 18] are the pioneers applying 3D convolutional neural networks on voxelized shapes. However, volumetric representation is constrained by its resolution due to data sparsity and computation cost of 3D convolution. FPNN [13] and Vote3D [26] proposed special methods to deal with the sparsity problem; however, their operations are still on sparse volumes, and it's challenging for them to process very large point clouds
> - **Multiview CNNs**: [23, 18] have tried to render 3D point clouds or shapes into 2D images and then apply 2D conv nets to classify them. With well-engineered image CNNs, this line of methods has achieved dominating performance on shape classification and retrieval tasks [21]. However,

> it's nontrivial to extend them to scene understanding or other 3D tasks such as point classification and shape completion
> - **Spectral CNNs**: Some latest works [4, 16] use spectral CNNs on meshes. However, these methods are currently constrained on manifold meshes such as organic objects, and it's not obvious how to extend them to non-isometric shapes such as furniture
> - **Feature-based DNNs**: [6, 8] firstly convert the 3D data into a vector, by extracting traditional shape features and then use a fully connected net to classify the shape. We think they are constrained by the representation power of the features extracted.

翻译:

3D数据的深度学习：3D 数据有多种流行的表示方式，因此出现了不同的学习方法。

- **体积卷积神经网络**：[28, 17, 18] 是最早将 3D 卷积神经网络应用于体素化形状的先驱。然而，体积表示由于数据稀疏性和 3D 卷积的计算成本而受限于分辨率。FPNN [13] 和 Vote3D [26] 提出了针对稀疏性问题的特殊方法；然而，它们的操作仍然是在稀疏体积上，处理非常大的点云依然具有挑战性。
- **多视图卷积神经网络**：[23, 18] 尝试将 3D 点云或形状渲染为 2D 图像，然后应用 2D 卷积网络进行分类。在经过精心设计的图像卷积网络支持下，这类方法在形状分类和检索任务中取得了优异的性能 [21]。然而，将它们扩展到场景理解或其他 3D 任务（如点分类和形状补全）并非易事。
- **谱卷积神经网络**：一些最新的研究 [4, 16] 在网格上使用谱卷积神经网络。然而，这些方法目前仅限于流形网格（如有机物体），如何将其扩展到非等距形状（如家具）尚不明确。
- **基于特征的深度神经网络**：[6, 8] 首先通过提取传统形状特征将 3D 数据转换为向量，然后使用全连接网络对形状进行分类。我们认为，它们受到提取特征的表示能力的限制。

解释:

- 这里对现有的对3D数据进行学习的方法进行了分析，并指出了它们的优缺点。体积卷积神经网络直接继承了2D图像的卷积操作，因而十分直观。但缺点也是显而易见的：体素占用空间过大。
- 多视图卷积神经网络则是将3D问题转化为传统的2D问题进行解决。事实上，传统的2D任务解决的也是现实的3维世界的问题，例如图像识别任务就是从3维现实世界采集单张2维图像进行识别。因而这也是一个比较直观的思路。但缺点也非常致命：它只能解决分类、检索等任务，和3D直接相关的任务无法解决。
- 谱卷积神经网络的“谱”应理解为“频谱”。在图像处理中，当空域操作较为困难时，可以转换到频域进行处理。类似地，虽然3维模型无法直接进行聚集，但Spectral CNNs [4] 通过在图拉普拉斯算子的特征值和特征向量（即频谱域）上定义卷积操作，将卷积扩展到非欧几里得空间。这种方法适用于**流形网格**（Manifold Meshes），即表面连续、无自交叠或裂缝的规则三维网格结构，常用于表示有机物体等流形模型。而**非等距形状**（Non-Isometric Shapes），即无法通过等距变换相互映射的形状，由于其复杂性和缺乏统一的几何结构，导致频谱特征不一致，因此无法直接使用谱卷积神经网络处理。这里读者理解到该方法适用范围窄即可。
- 基于特征的深度神经网络的优势在于结构简单，但缺点也十分明显：对特征设计的要求较高。总的来说，上述四种方法都或多或少地存在问题。

> **Deep Learning on Unordered Sets**: From a data structure point of view, a point cloud is an unordered set of vectors. While most works in deep learning focus on regular input representations like sequences (in speech and language processing), images, and volumes (video or 3D data), not much work has been done in deep learning on point sets.

> One recent work from Oriol Vinyals et al. [25] looks into this problem. They use a read-process-write network with an attention mechanism to consume unordered input sets and show that their network has the ability to sort numbers. However, since their work focuses on generic sets and NLP applications, it lacks the consideration of geometry in the sets.

翻译:

针对无序集合的深度学习：从数据结构的角度来看，点云是一个无序的向量集合。尽管深度学习的许多研究集中在序列（如语音和语言处理）、图像以及体积（如视频或三维数据）等规则输入表示上，但在点集的深度学习研究方面尚未有太多工作。

Oriol Vinyals 等人最近的一项研究 [25] 探讨了这个问题。他们使用了带有注意力机制的读-处理-写网络来处理无序输入集合，并展示了该网络具有对数字进行排序的能力。然而，由于他们的工作主要集中在通用集合和自然语言处理应用上，因此对集合中的几何特性考虑不足。

解释:

- 在作者所处的2017年，针对无序集合的研究工作相对较少。就深度学习最热门的两大任务——图像处理和自然语言处理而言，图像具有规则性和有序性，语言则是自相关的序列结构。它们都与无序集合存在较大差异。

---

3. Problem Statement（问题陈述）

> We design a deep learning framework that directly consumes unordered point sets as inputs. A point cloud is represented as a set of 3D points $\{P_i|\ i=1,\ldots,n\}$, where each point P_i is a vector of its (x,y,z) coordinate plus extra feature channels such as color, normal, etc. For simplicity and clarity, unless otherwise noted, we only use the (x,y,z) coordinate as our point's channels.

翻译:

我们设计了一个深度学习框架，能够直接处理无序的点集作为输入。点云表示为一组3D点 $\{P_i|\ i=1,\ldots,n\}$ ，其中每个点 P_i 是其 (x,y,z) 坐标的向量，外加额外的特征通道，如颜色、法线等。为简明起见，除非另有说明，我们仅使用 (x,y,z) 坐标作为点的通道。

> For the object classification task, the input point cloud is either directly sampled from a shape or pre-segmented from a scene point cloud. Our proposed deep network outputs k scores for all the k candidate classes. For semantic segmentation, the input can be a single object for part region segmentation, or a sub-volume from a 3D scene for object region segmentation. Our model will output $n\times m$ scores for each of the n points and each of the m semantic sub-categories.

翻译:

对于物体分类任务，输入的点云可以直接从形状中采样，也可以从场景点云中预先分割而来。我们提出的深度网络为所有 k 个候选类别输出 k 个评分。对于语义分割，输入可以是用于部件区域分割的单个物体，或者是用于物体区域分割的3D场景中的子体积。我们的模型将为每个 n 个点和每个 m 个语义子类别输出 $n\times m$ 个评分。

解释:

- 笔者认为，读者如果对深度学习有基本的理解，知道深度学习如何表示并学习分类、语义分割问题，应该都能够理解上面的两段话。“输入可以是用于部件区域分割的单个物体”指输入单个物体的点云，如一个台灯，然后将其分割为各个组件。“用于物体区域分割的3D场景中的子体积”指将整个场景拆分成较小的局部区域，逐块按不同物体进行分割。

---

4. Deep Learning on Point Sets（针对点集的深度学习）

> The architecture of our network (Sec 4.2) is inspired by the properties of point sets in $\mathbb{R}^n$ (Sec 4.1).

翻译：我们网络的架构（第4.2节）受到$\mathbb{R}^n$中点集特性的启发（第4.1节）。

4.1 Properties of Point Sets in $\mathbb{R}^n$

> Our input is a subset of points from an Euclidean space. It has three main properties:
>
> - **Unordered.** Unlike pixel arrays in images or voxel arrays in volumetric grids, point cloud is a set of points without specific order. In other words, a network that consumes N 3D point sets needs to be invariant to $N!$ permutations of the input set in data feeding order.
> - **Interaction among points.** The points are from a space with a distance metric. It means that points are not isolated, and neighboring points form a meaningful subset. Therefore, the model needs to be able to capture local structures from nearby points, and the combinatorial interactions among local structures.
> - **Invariance under transformations.** As a geometric object, the learned representation of the point set should be invariant to certain transformations. For example, rotating and translating points all together should not modify the global point cloud category nor the segmentation of the points.

翻译：

我们的输入是欧几里得空间中若干点构成的子集。它具有三个主要特性：

- **无序性**。与图像中的像素阵列或体素网格中的体素阵列不同，点云是一组没有特定顺序的点集。换句话说，一个处理N个三维点集的网络，需要在数据输入顺序的$N!$种排列变换下保持不变性。
- **点间相互作用**。这些点来自具有距离度量的空间。这意味着点并非孤立存在，相邻的点会构成有意义的子集。因此，模型需要能够从邻近点中捕捉局部结构，以及局部结构之间的组合相互作用。
- **变换下的不变性**。作为一种几何对象，点集经过学习得到的表示应当在某些变换下保持不变。例如，将所有点一起旋转和平移，不应改变整个点云的类别，也不应改变点的分割情况。

解释：

- 无序性和不变性在前文已经进行了说明。
- 第二点说明了提取局部特征的必要性。语义之所以能够存在，就是因为一个集合或一个序列中的各个元素不是互相孤立的，而是相互联系的。正是这些联系使它们产生了意义。点云虽然是无序的，但不同点之间存在空间上的近邻关系，这为提取局部信息提供了基础。

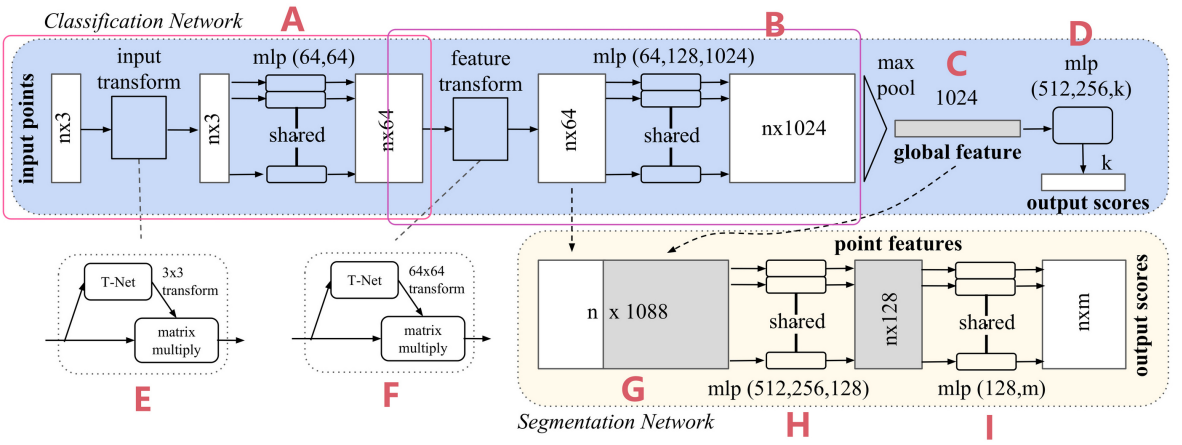


Figure 2. PointNet Architecture. The classification network takes n points as input, applies input and feature transformations, and then aggregates point features by max pooling. The output is classification scores for k classes. The segmentation network is an extension to the classification net. It concatenates global and local features and outputs per point scores. “mlp” stands for multi-layer perceptron, numbers in bracket are layer sizes. Batchnorm is used for all layers with ReLU. Dropout layers are used for the last mlp in classification net.

图2. PointNet架构。分类网络以n个点作为输入，应用输入和特征变换，然后通过最大池化聚合点特征。输出是k个类别的分类得分。分割网络是分类网络的扩展。它连接全局和局部特征，并输出每个点的得分。“mlp”代表多层感知器，括号中的数字是层的大小。所有层都使用批归一化和ReLU激活函数。分类网络最后的mlp使用了Dropout层。

4.2 PointNet Architecture（PointNet架构）

> Our full network architecture is visualized in Fig 2, where the classification network and the segmentation network share a great portion of structures. Please read the caption of Fig 2 for the pipeline.
>
> Our network has three key modules: the max pooling layer as a symmetric function to aggregate information from all the points, a local and global information combination structure, and two joint alignment networks that align both input points and point features.
>
> We will discuss our reason behind these design choices in separate paragraphs below.

翻译:

我们的完整网络架构在图2中进行了可视化，其中分类网络和分割网络共享了很大一部分结构。请参阅图2的说明以了解整个流程。

我们的网络有三个关键模块：作为对称函数的最大池化层，用于从所有点聚合信息；局部和全局信息组合结构；以及两个联合对齐网络，用于对齐输入点和点特征。

我们将在下面的段落中分别讨论这些设计选择背后的原因。

解释:

- 正如论文在前文中说的，这个网络的结构“surprisingly simple”，没有用到任何复杂的组件。如果您通过前文已经了解了各个主要步骤的原理，相信这个架构图也不难理解。这里我们对整个网络的流程进行梳理。笔者一般习惯于通过追踪信息流动的方式梳理流程，即通过每一步的输入、输出及其格式将各个部分串联起来。
- A部分可以看做是一个整体。
  - 我们首先有一个点云作为输入，其维度为$(n, 3)$，其中n为点的数量，每个点共x, y, z三个分量。
  - 在正式送入**MLP（多层感知机）**进行处理前，这个输入会首先送到T-Net中。T-Net输入一个维度为$(n, 3)$的点云，输出一个3×3的变换矩阵（对变换矩阵不太熟悉的读者可以参考这篇文章，它的功能是对点进行平移、旋转等操作）。
  - 接下来让这个变换矩阵作用于输入的点云，得到对齐后的点云，此时维度仍是$(n, 3)$。
  - 将对齐的点云送入MLP得到输出，输出维度为$(n, 64)$，这个输出可以看作是点云的低级特征，如几何特征。此处MLP标了一个“shared”，即共享权重。这说明对每个点应用的是相同的MLP网络，而不是对每个点单独训练网络，这使得PointNet能够适应任意大小的点云输入。
- B部分也可以看作是一个整体，它的结构和A几乎一样，只是输入变成了$(n, 64)$的特征向量组合。B部分输出维度为$(n, 1024)$的特征向量组合，反映了更多高级特征。这一步还对特征空间进行了一次对齐，仍使用T-Net，这次T-Net输出的是64×64的变换矩阵。
- C部分是Max Pooling操作。这一步在前文已经讲解过了。它输出$(1, 1024)$的一个特征向量。
- 其他部分则是具体任务的应用了。D部分是针对分类任务的，它将维度为$(1, 1024)$的特征向量变换为k各分类的得分，用于最终的判断。
- G、H、I对应的是语义分割任务。G处首先将维度为$(1, 1024)$的特征向量复制n份，形成$(n, 1024)$的矩阵。然后和维度为$(n, 64)$的中间输出进行拼接，形成$(n, 1088)$的大矩阵。此时，每个点对应的特征向量共1088维，它既包含了64维的局部低级特征，又包含了1024维的全局高级特征，我们通过H对这两部分信息进行整合，然后通过I输出每个点云所属的类别。语义分割任务和分类任务的不同在于，分类任务仅关心全局的、整体的特征，而点云语义分割任务需要为每个点分配类别，这不仅要求了解全局上下文（整体语义信息），还需要保留局部特征，以捕捉点与其邻域的细粒度结构信息。因此，语义分割需要拼接局部特征和全局特征来同时考虑局部几何和整体语义。

> **说明**：到这里我们就已经对PointNet的整体架构和基本原理有了一个基本的、全面的认识，如果您只是想在自己的项目中应用PointNet，希望对其架构、原理进行基本的了解，那么到这里就已经足够了，您可以直接跳到实验或结论部分。但如果您对PointNet的原理感兴趣，或者希望基于PointNet进行学术研究，笔者建议您阅读完剩余的部分，这部分包含理论分析和大量数学计算，建议读者静心阅读、反复思考。

> **Symmetry Function for Unordered Input**
>
> In order to make a model invariant to input permutation, three strategies exist:
>
> 1. Sort input into a canonical order.
> 2. Treat the input as a sequence to train an RNN, but augment the training data by all kinds of permutations.
> 3. Use a simple symmetric function to aggregate the information from each point.
>
> Here, a symmetric function takes n vectors as input and outputs a new vector that is invariant to the input order. For example, $+$ and $*$ operators are symmetric binary functions.

翻译:

适用于无序输入的对称函数

为了使模型对输入的排列不变性（permutation invariant）具有鲁棒性，通常有以下三种策略：

1. 将输入排序为一种规范顺序；
2. 将输入当作序列并训练一个 RNN，同时通过各种排列对训练数据进行数据增强；
3. 使用一种简单的对称函数来汇总每个点的信息。

这里，对称函数接受 n 个向量作为输入，并输出一个对输入顺序不敏感的新向量。例如，$+$ 和 $\times$ 运算符是对称的二元函数。

解释:

- 这里作者开始对PointNet关键部分的原理进行介绍。为帮助读者对PointNet有一个初步的认识，前文已经涉及到这部分内容，这里我们进行更加深入的解释。
- 首先，作者需要解决“无序输入”的问题，即让PointNet实现**置换不变性**（permutation invariance）。作者列出了三种可能的解决方案。首先是使用一定的规则对点云进行排序，这个方法要求无论如何打乱原来的序列，算法排序后的结果都是保持一致，相关内容作者在下一段进行了论述。
- 第二种则是通过随机打乱顺序进行输入，使得神经网络能够适应任意顺序的输入。
- 第三种则是作者使用的方法。这里再介绍一下什么是对称函数。作者提到了加法和乘法，我们小学四年级就学过，加法和乘法是具有交换律的，例如：$a+b+c=b+c+a=a+c+b$，也就是输出和输入的顺序无关。一个反例是减法，它显然是非对称的。

定义:

设 $f:(X^n)\to Y$ 是一个函数，其输入是 n 个来自集合 X 的向量，输出是集合 Y 中的一个元素。如果对于任意 n 个相同的输入向量 $v_1,v_2,\ldots,v_n\in X$ 和它们的任意排列 $\sigma\in S_n$（其中 S_n 是所有 n 个元素的排列的集合），有：

$$f(v_1,v_2,\ldots,v_n)=f(v_{\sigma(1)},v_{\sigma(2)},\ldots,v_{\sigma(n)}),$$

则称 f 是一个**对称函数。**

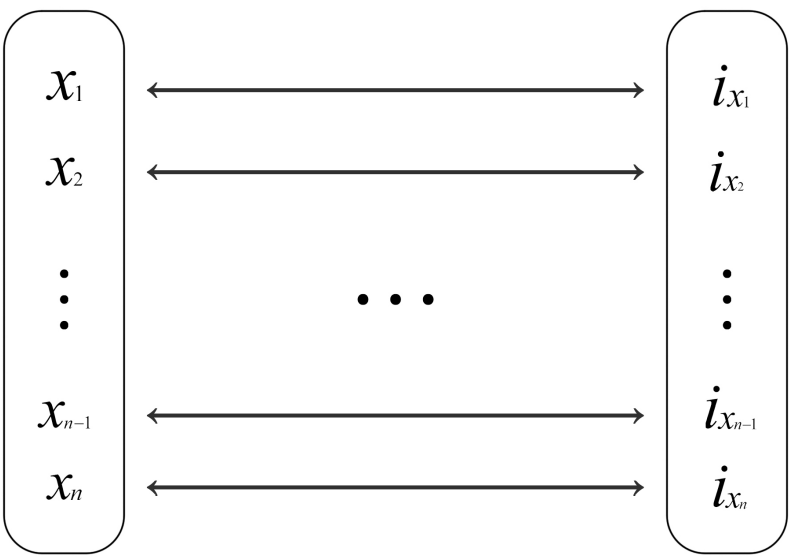
> While sorting sounds like a simple solution, in high-dimensional space there does not exist an ordering that is stable with respect to point perturbations in the general sense. This can be easily shown by contradiction. If such an ordering strategy exists, it defines a bijection map between a high-dimensional space and a $1d$ real line. It is not hard to see that requiring an ordering to be stable with respect to point perturbations is equivalent to requiring that this map preserves spatial proximity as the dimension reduces, a task that cannot be achieved in the general case. Therefore, sorting does not fully resolve the ordering issue, and it's hard for a network to learn a consistent mapping from input to output as the ordering issue persists. As shown in experiments (Fig 5), we find that applying a MLP directly on the sorted point set performs poorly, though slightly better than directly processing an unsorted input.

翻译:

① 尽管排序看起来是一个简单的解决方案，但在高维空间中，并不存在一种对点扰动（point perturbations）稳定的排序方法。② 这可以通过反证法轻松证明。如果这样一种排序策略存在，它将定义一个**高维空间**与 $1d$ **实数线**之间的**双射映射**。③ 而很容易看出，要求这种排序对点扰动稳定相当于要求这种映射在降维过程中保持空间的接近性（spatial proximity），④ 而在一般情况下这是无法实现的。因此，排序并不能完全解决排列问题，⑤ 并且由于排列问题的存在，神经网络难以学习输入到输出的一致映射。⑥ 正如实验中所示（图 5），我们发现直接对排序后的点集应用 MLP 表现较差，但比直接处理未排序输入稍好。

解释：

- 这一段解释了为什么难以找到一种稳定的算法对点云进行排序。这里首先说明什么是“对点扰动稳定”。排序的“稳定性”是指如果点云中的点发生了轻微扰动（例如由于噪声或测量误差），排序的结果也应该只发生轻微变化，而不是完全乱掉，这是我们对一个排序算法最基本的期待。
- ②③④句作者用几句话论述了为什么这样一个排序算法是无法找到的，但作者说得比较模糊。下面笔者尝试用更加精确的语言进行分析。由于笔者水平原因，下面的数学论述可能有错误，而且并没有直接证明该结论，但笔者希望提供一个分析的思路供读者理解。
- 首先，我们说明“定义一个高维空间与 $1d$ 实数线之间的双射映射”是什么意思。
  - 这里我们首先思考这样一个问题：我们大一就学习了各种排序算法，但如何统一描述这些算法呢？这里我指的是所有种类的排序算法，与其实现原理无关。如果使用输入输出来描述排序算法的话，排序算法可以抽象为：输入一个n个元素的顺序数组$[x_1, x_2, \ldots, x_n]$，输出一个同等长度的数组$[i_{x_1}, i_{x_2}, \ldots, i_{x_n}]$，其中$i_{x_n}$代表$x_n$在排序后数组的索引。例如，输入$[8, 10, 5]$，则输出$[2, 3, 1]$（假设从1开始计数）。注意，特定的元素总是对应一个确定的输出，例如，输入$[5, 10, 8]$则输出$[1, 3, 2]$，5总是对应1，10总是对应3。
  - 作者所说的“高维空间”指的就是x_n的维度。排序算法中输入的顺序数组的元素类型是任意的，对应到点云排序这个具体任务，x_n的格式就是一个数组，我们设它为d维，对应一个点的坐标或一个点的特征向量。
  - 而排序算法的输出i_{x_n}显然一定是一个整数。当然，如果我们扩展下定义，输出实数也是可以的，只要能够按大小排出先后即可。这些数都排列在一个1维的数轴上，这正是原文说的“$1d$ 实数线”。
  - 前面提到，特定的元素总是对应一个确定的输出，不管输入的顺序如何。不管输入是$[x_2, x_1, \ldots, x_n]$还是$[x_n, x_{n-1}, \ldots, x_1]$，$x_1$都一定对应$i_{x_1}$，$x_n$都一定对应$i_{x_n}$，反过来也成立。因此，我们就可以构建这样一个双射映射：



其中，x_n代表一个d维向量；i_{x_n}代表该点的排序序号，是一个实数。这就是原文所说的“定义一个高维空间与 $1d$ 实数线之间的双射映射”。

- 接下来，我将用自己的思路分析③④的结论。
  - 首先，我们给点集做一个定义。设一个高维空间中的点集用矩阵 $X \in \mathbb{R}^{n\times d}$ 表示，其中n是点的数量，d代表空间的维度，可以是三维空间，也可以是更高维的特征空间。
  - 从高维空间映射到低维空间也叫**投影**（projection）。这里我们假设投影是线性的，从高维空间 $\mathbb{R}^d$ 映射到低维空间 $\mathbb{R}^k$ 的过程可以描述为：

    $$Y = XW, \quad W \in \mathbb{R}^{d\times k},$$

    其中 $Y \in \mathbb{R}^{n\times k}$ 是投影后的点集，W 是投影矩阵。
- 上面我们对问题进行了更加严谨、清晰地定义。不过由于笔者水平原因，无法继续给出严谨的证明，当然论文作者也没有。不过为了方便读者理解、感受为什么找到这样一个排序算法是困难的，我们可以继续做如下思考：
  - 找到一个排序算法，其实可以转换为这样一个问题，即对于$Y = XW$，是否存在$V \in \mathbb{R}^{k\times d}$，使得$X = YV$，即从高维空间通过$W$映射到低维空间后，不会产生多对一情况，可以通过$V$唯一地映射回去。根据实际情况，这里$k = 1, d > k$。这里不能使用逆矩阵的概念，而应该使用**伪逆**（pseudoinverse）。
  - 具体过程太复杂就不写了，但根据伪逆的存在性定理和实际情况，这个V应该是不存在的。也就是说，上面说的双射实际构建不出来。
  - 如果我们摆脱线性变换这个条件的话，能否构建出这样的双射呢？结合我浅薄的数学知识，我认为应该是可以的，**因为$\mathbb{R}^n$空间中点的数量和$\mathbb{R}$中点的数量（即实数的数量）是一样的，且可以为每个点构造一个双射。**证明过程可以参照这个网页，按照该网页提供的构造方法，我们是可以构造一个函数，将每个点云的坐标或特征向量映射为一个实数，并用它来排序的。
  - 然而，一个好的排序算法除了正确还应该具有稳定性，即点的位置（包括在特征空间中的位置）发生轻微扰动后，映射到实数轴上的位置大致不变。换句话说，高维空间中相近的点在映射后仍然相近，即原文④句所说。至于是否能构造出这样一个算法，或能否通过神经网络学习出这样一个算法，这里就无法继续深入了。作者⑥中的实验也表明进行排序效果并不好。

> The idea to use RNN considers the point set as a sequential signal and hopes that by training the RNN with randomly permuted sequences, the RNN will become invariant to input order. However, in "OrderMatters" [25], the authors have shown that order does matter and cannot be totally omitted. While RNNs have relatively good robustness to input ordering for sequences with small lengths (dozens), it's hard to scale to thousands of input elements, which is the common size for point sets. Empirically, we have also shown that models based on RNN do not perform as well as our proposed method (Fig 5).

翻译：

使用 RNN 的方法将点集视为一个序列信号，希望通过用随机排列的序列训练 RNN，使 RNN 对输入顺序具有不变性。然而，在 "OrderMatters" [25] 一文中，作者表明顺序确实重要，且无法完全忽略。尽管 RNN 对于小长度序列（几十个点）的输入顺序具有较好的鲁棒性，但很难扩展到数千个输入元素，这在点集处理中相当常见。从经验上看，我们也表明基于 RNN 的模型性能不如我们提出的方法（图 5）。

解释：

- 这一段更多地是从经验入手进行分析。对于点云这种大规模数据，RNN似乎难以展现出对输入顺序的鲁棒性，即使使用数据增强等手段进行训练。

> Our idea is to approximate a general function defined on a point set by applying a symmetric function on transformed elements in the set:
>
> $$f(\{x_1, \ldots, x_n\}) \approx g(h(x_1), \ldots, h(x_n)),$$
>
> where $f : 2^{\mathbb{R}^N} \to \mathbb{R}$, $h : \mathbb{R}^N \to \mathbb{R}^K$, and $g : \underbrace{\mathbb{R}^K \times \cdots \times \mathbb{R}^K}_{n} \to \mathbb{R}$ is a symmetric function.
>
> Empirically, our basic module is very simple: we approximate h by a multi-layer perceptron network and g by a composition of a single variable function and a max pooling function. This is found to work well by experiments. Through a collection of h, we can learn a number of f's to capture different properties of the set.

翻译：

我们的想法是，通过对点集中每个元素的变换应用**对称函数**，来近似一个定义在点集上的广义函数：

$$f(\{x_1, \ldots, x_n\}) \approx g(h(x_1), \ldots, h(x_n)),$$

其中 $f : 2^{\mathbb{R}^N} \to \mathbb{R}$，$h : \mathbb{R}^N \to \mathbb{R}^K$，而 $g : \underbrace{\mathbb{R}^K \times \cdots \times \mathbb{R}^K}_{n} \to \mathbb{R}$ 是一个对称函数。

从经验上看，我们的基本模块非常简单：我们用一个多层感知机（MLP）网络来近似 h，用单变量函数与最大池化（max pooling）函数的组合来近似 g。实验表明这种方法效果很好。① 通过一组 h 函数，我们可以学习多个 f 函数，从而捕获点集的不同属性。

解释：

- 这一部分使用了数学定义，看起来很难懂，但其实掌握其实际意义后就不难理解了。f是一个点集函数，它输入一个集合，集合的每个元素都是一个N维的向量，$2^{\mathbb{R}^N}$ 是 $\mathbb{R}^N$ 的幂集，表示所有可能的点集（无序集合）。输出维度为什么是$\mathbb{R}$我不太理解，它应该是对每个维度进行聚合的，输出应该是一个向量？这里只能暂时存疑。
- 而PointNet做的，就是使用一个神经网络去拟合这个理论上的f函数，这里作者用g表示这个近似的函数。其中x_i表示第i个点，共N维；h函数对应网络中的MLP，而g函数对应网络中的Max Pooling。
- ①是说，在g保持不变的情况下（使用Max Pooling），学习不同的h，函数复合后就近似不同的f，这就可以用于多个下游任务。由此看来，完成不同下游任务时基本网络的结构相同，但参数不相同，需要重新针对任务进行训练。

> While our key module seems simple, it has interesting properties (see Sec 5.3) and can achieve strong performance (see Sec 5.1) in a few different applications. Due to the simplicity of our module, we are also able to provide theoretical analysis as in Sec 4.3.

翻译：

虽然我们的关键模块看起来简单，但它具有一些有趣的特性（见 5.3 节），并且在几种不同的应用中表现出色（见 5.1 节）。由于模块简单，我们还能够提供理论分析（见 4.3 节）。

> **Local and Global Information Aggregation**
>
> The output from the above section forms a vector $[f_1, \ldots, f_K]$, which is a global signature of the input set. We can easily train a SVM or multi-layer perceptron classifier on the shape global features for classification. However, point segmentation requires a combination of local and global knowledge. We can achieve this in a simple yet highly effective manner.

翻译：

局部和全局信息的聚合

上述部分的输出形成一个向量 $[f_1, \ldots, f_K]$，这是输入集的全局特征。我们可以轻松地在形状全局特征上训练支持向量机（SVM）或多层感知器分类器进行分类。然而，点分割需要局部和全局知识的结合。我们可以通过一种简单而高效的方法来实现这一点。

解释：

-

> Our solution can be seen in Figure 2 (Segmentation Network). After computing the global point cloud feature vector, we feed it back to per-point features by concatenating the global feature with each of the point features. Then we extract new per-point features based on the combined point features—this time the per-point feature is aware of both the local and global information.

翻译：

我们的解决方案可以在图2（分割网络）中看到。在计算全局点云特征向量后，我们通过将全局特征与每个点特征连接，将其反馈到每个点特征中。然后，我们基于组合后的点特征提取新的每点特征——这次每点特征能够意识到局部和全局信息。

> With this modification, our network is able to predict per-point quantities that rely on both local geometry and global semantics. For example, we can accurately predict per-point normals (see figure in supplementary materials), validating that the network is able to summarize information from the point's local neighborhood. In the experimental section, we also show that our model can achieve state-of-the-art performance on shape part segmentation and scene segmentation.

翻译：

通过这一修改，我们的网络能够预测依赖于局部几何和全局语义的每点量。例如，我们可以准确预测每点法线（参见补充材料中的图），验证网络能够总结点的局部邻域信息。在实验部分，我们还展示了我们的模型在形状部件分割和场景分割方面能够实现最先进的性能。

Joint Alignment Network

The semantic labeling of a point cloud must be invariant under certain geometric transformations, such as rigid transformations. Therefore, we expect that the learned representation by our point set is invariant to these transformations.

A natural solution is to align all input sets to a canonical space before feature extraction. Jaderberg et al. [9] introduced the idea of a spatial transformer to align 2D images through sampling and interpolation, achieved by a specifically tailored layer implemented on GPU.

Our input form of point clouds allows us to achieve this goal in a much simpler way compared with [9]. We do not need to invent any new layers, and no aliasing is introduced as in the image case. We predict an affine transformation matrix by a mini-network (T-net in Fig 2) and directly apply this transformation to the coordinates of the input points. The mini-network itself resembles the larger network and is composed of basic modules for point-independent feature extraction, max pooling, and fully connected layers. More details about the T-net are in the supplementary.

This idea can be further extended to the alignment of feature space as well. We can insert another alignment network on point features and predict a feature transformation matrix to align features from different input point clouds. However, the transformation matrix in the feature space has a much higher dimension than the spatial transform matrix, which greatly increases the difficulty of optimization. Therefore, we add a regularization term to our softmax training loss, constraining the feature transformation matrix to be close to an orthogonal matrix:

$$L_{reg} = \|I - AA^T\|_F^2,$$

where A is the feature alignment matrix predicted by a mini-network. An orthogonal transformation will not lose information in the input, which is desirable. We find that by adding the regularization term, the optimization becomes more stable, and our model achieves better performance.

联合对齐网络

点云的语义标注必须对某些几何变换（如刚性变换）保持不变。因此，我们期望通过我们的点集学习到的表示对于这些变换是不变的。

一种自然的解决方案是在特征提取之前将所有输入集对齐到一个规范空间。Jaderberg 等人 [9] 提出了通过采样和插值对 2D 图像进行对齐的空间变换器的概念，这通过在 GPU 上实现的专门设计的层来实现。

我们点云的输入形式使我们能够以比 [9] 更简单的方式实现这个目标。我们无需发明任何新层，也不会像图像那样引入别名效应。我们通过一个小型网络（图 2 中的 T-net）预测一个仿射变换矩阵，并直接将该变换应用于输入点的坐标。该小型网络本身类似于更大的网络，由点无关特征提取、最大池化和全连接层的基本模块组成。关于 T-net 的更多细节请参见补充材料。

这个想法可以进一步扩展到特征空间的对齐。我们可以在点特征上插入另一个对齐网络，并预测一个特征变换矩阵，以对齐来自不同输入点云的特征。然而，特征空间中的变换矩阵的维度远高于空间变换矩阵，这大大增加了优化的难度。因此，我们在softmax训练损失中添加了一个正则化项，约束特征变换矩阵接近正交矩阵：

$$L_{reg} = \|I - AA^T\|_F^2,$$

其中 A 是由小型网络预测的特征对齐矩阵。正交变换不会丢失输入中的信息，这是理想的。我们发现，通过添加正则化项，优化变得更加稳定，我们的模型表现也得到了提升。